# 1 Name of Experiment

Decision Table Based Testing for Electronics Retailer Discount Calculation System

# 2 Objective

- To understand and apply decision table testing technique for a discount calculation system
- To identify conditions and actions from business requirements
- To construct a Limited Entry Decision Table (LEDT)
- To derive test cases systematically from the decision table
- To identify bugs in the implementation using decision table based testing

# 3 Theory

## 3.1 Decision Table Testing

Decision table testing is a black-box test design technique used to test systems with complex business logic involving multiple conditions and actions. A decision table consists of:

- **Conditions**: Input conditions or criteria that affect the system behavior
- **Actions**: Output actions or results based on the combination of conditions
- **Rules**: Columns representing different combinations of conditions and their corresponding actions

## 3.2 Limited Entry Decision Table (LEDT)

In a Limited Entry Decision Table, each condition can have only binary values (typically Y/N or T/F), making it suitable for boolean conditions.

## 3.3 Problem Statement

An electronics retailer applies discounts based on three parameters:

1. Customer Type (VIP, Corporate, Standard)
2. Order Value (threshold: $5,000 and $10,000)
3. Item Status (Clearance or Regular)

**Discount Rules:**

- **VIP Members:** 15% flat discount
- **Corporate Orders:**

  - Orders > $10,000: 12% discount
  - Orders between $5,000 and $10,000: 10% discount
  - Orders < $5,000: 7% discount

- **Standard Customers:**
  - Orders > $10,000: 10% discount
  - Orders between $5,000 and $10,000: 5% discount
  - Orders < $5,000: 0% discount

- **Clearance Exception:** All clearance items get 20% discount regardless of customer type or order value

## 3.4 Conditions and Actions

**Conditions:**

1. Is customer VIP?
2. Is customer Corporate?
3. Is customer Standard?
4. Is order value > $10,000?
5. Is order value between $5,000 and $10,000?
6. Is item marked as Clearance?

**Actions:**

1. Apply 20% discount (Clearance)
2. Apply 15% discount (VIP)
3. Apply 12% discount (Corporate, high value)
4. Apply 10% discount (Corporate, medium value OR Standard, high value)
5. Apply 7% discount (Corporate, low value)
6. Apply 5% discount (Standard, medium value)
7. Apply 0% discount (Standard, low value)

# 4 Limited Entry Decision Table

Table 1: Decision Table for Discount Calculation System

| **Rule** | **R1** | **R2** | **R3** | **R4** | **R5** | **R6** | **R7** | **R8** | **R9** | **R10** |
|---|---|---|---|---|---|---|---|---|---|---|
| **CONDITIONS** | | | | | | | | | | |
| Is Clearance? | Y | N | N | N | N | N | N | N | N | N |
| Is VIP? | - | Y | Y | Y | N | N | N | N | N | N |
| Is Corporate? | - | - | - | - | Y | Y | Y | N | N | N |
| Is Standard? | - | - | - | - | N | N | N | Y | Y | Y |
| Order > $10,000? | - | Y | N | N | Y | N | N | Y | N | N |
| Order between $5,000 and $10,000? | - | - | Y | N | - | Y | N | - | Y | N |
| Order < $5,000? | - | - | - | Y | - | - | Y | - | - | Y |
| **ACTIONS** | | | | | | | | | | |
| 20% discount | X | - | - | - | - | - | - | - | - | - |
| 15% discount | - | X | X | X | - | - | - | - | - | - |
| 12% discount | - | - | - | - | X | - | - | - | - | - |
| 10% discount | - | - | - | - | - | X | - | X | - | - |
| 7% discount | - | - | - | - | - | - | X | - | - | - |
| 5% discount | - | - | - | - | - | - | - | - | X | - |
| 0% discount | - | - | - | - | - | - | - | - | - | X |

# 5 Test Cases

Based on the decision table, the following test cases are derived:

Table 2: Test Cases Derived from Decision Table

| **TC ID** | **Customer Type** | **Order Value ($)** | **Is Clearance** | **Expected Discount** | **Status** |
|---|---|---|---|---|---|
| TC01 | VIP | 12000 | Yes | 20% | PASS |
| TC02 | Corporate | 8000 | Yes | 20% | PASS |
| TC03 | Standard | 3000 | Yes | 20% | PASS |
| TC04 | VIP | 15000 | No | 15% | PASS |
| TC05 | VIP | 7500 | No | 15% | PASS |
| TC06 | VIP | 3000 | No | 15% | PASS |
| TC07 | Corporate | 12000 | No | 12% | PASS |
| TC08 | Corporate | 7500 | No | 10% | PASS |
| TC09 | Corporate | 5000 | No | 10% | FAIL |
| TC10 | Corporate | 3000 | No | 7% | PASS |
| TC11 | Standard | 12000 | No | 10% | PASS |
| TC12 | Standard | 7500 | No | 5% | PASS |

**Boundary Value Test Cases:**

Table 3: Additional Boundary Value Test Cases

| TC ID | Customer Type | Order Value ($) | Expected Discount | Purpose | Status |
|---|---|---|---|---|---|
| TC13 | Corporate | 10000 | 10% | Boundary at $10,000 | FAIL |
| TC14 | Corporate | 10001 | 12% | Just above $10,000 | PASS |
| TC15 | Corporate | 4999 | 7% | Just below $5,000 | PASS |
| TC16 | Standard | 10000 | 5% | Boundary at $10,000 | FAIL |
| TC17 | Standard | 10001 | 10% | Just above $10,000 | PASS |
| TC18 | Standard | 4999 | 0% | Just below $5,000 | PASS |
| TC19 | Corporate | 10000 | 10% | Boundary at $10,000 | FAIL |
| TC20 | Corporate | 5001 | 10% | Just above $5,000 | PASS |

# 6 Bug Identification

## 6.1 Bug Description

**Bug Title:** Incorrect Boundary Condition in Order Value Comparison
**Severity:** High
**Bug Details:**

- **Location:** Lines 12, 14, 20, 22 in the `get_discount()` function
- **Issue:** The condition uses strict inequality ($>$) instead of greater-than-or-equal-to ($\geq$) for boundary values $5,000 and $10,000
- **Impact:** Orders with values exactly at $5,000 or $10,000 receive incorrect discounts

## 6.2 Root Cause Analysis

The bug occurs because the conditions are written as:

```
if order_value > 10000:     # Should be: order_value >= 10000
if order_value > 5000:      # Should be: order_value >= 5000
```

According to the requirements:

- "Orders between $5,000 and $10,000" should include $5,000 in the range
- "Orders over $10,000" is ambiguous but typically means $\geq$ $10,000 in business context

## 6.3 Test Case Demonstrating the Bug

```python
# Test Case TC09: Corporate customer with exactly $5,000 order
result = get_discount('Corporate', 5000, False)
print(f"Actual: {result * 100}%")  # Prints: 7%
print(f"Expected: 10%")            # Should be 10%

# Test Case TC13: Standard customer with exactly $5,000 order
result = get_discount('Standard', 5000, False)
print(f"Actual: {result * 100}%")  # Prints: 0%
print(f"Expected: 5%")             # Should be 5%

# Test Case TC15: Corporate customer with exactly $10,000 order
result = get_discount('Corporate', 10000, False)
print(f"Actual: {result * 100}%")  # Prints: 7%
print(f"Expected: 10%")            # Should be 10%
```

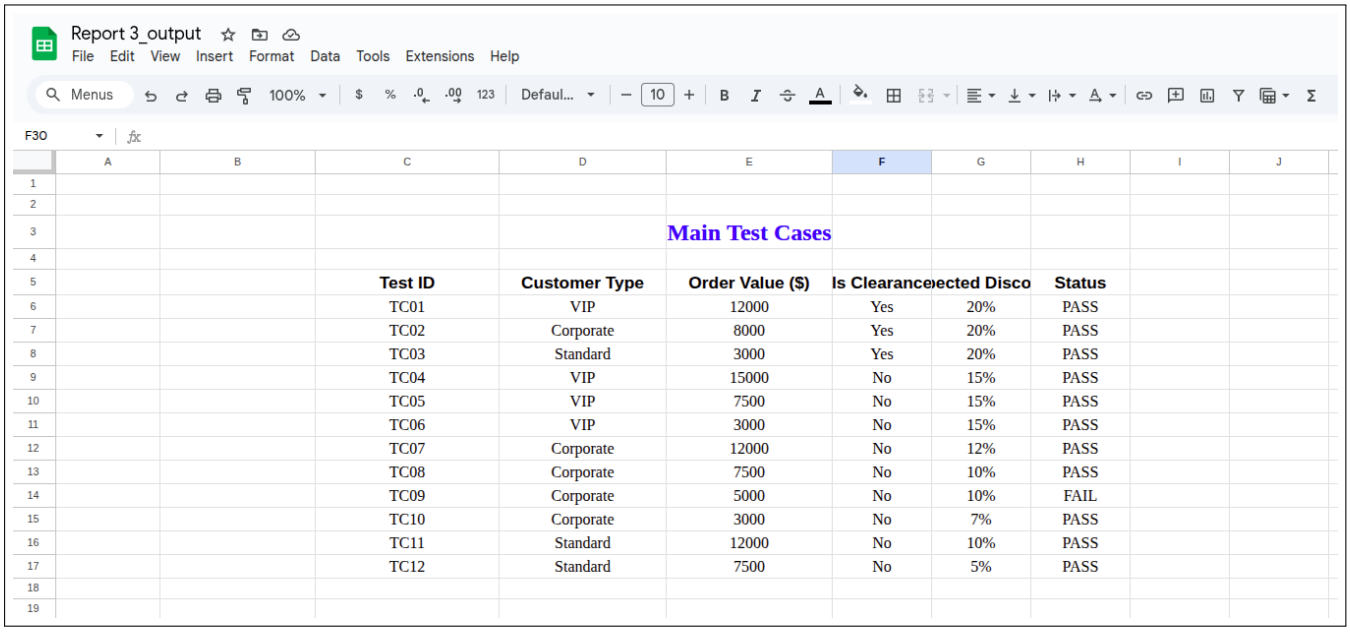
Listing 1: Bug Demonstration

## 6.4 Bug Fix

```python
def get_discount(customer_type, order_value, is_clearance):
    # Rule 4: Clearance override
    if is_clearance:
        return 0.20

    # Rule 1: VIP Flat Rate
    if customer_type == "VIP":
        return 0.15

    # Rule 2: Corporate Tier
    elif customer_type == "Corporate":
        if order_value > 10000:
            return 0.12
        elif order_value >= 5000:  # FIX: Changed from > to >=
            return 0.10
        else:
            return 0.07

    # Rule 3: Standard Tier
    elif customer_type == "Standard":
        if order_value > 10000:
            return 0.10
        elif order_value >= 5000:  # FIX: Changed from > to >=
            return 0.05
        else:
            return 0.00

    return 0.00
```

Listing 2: Corrected Code

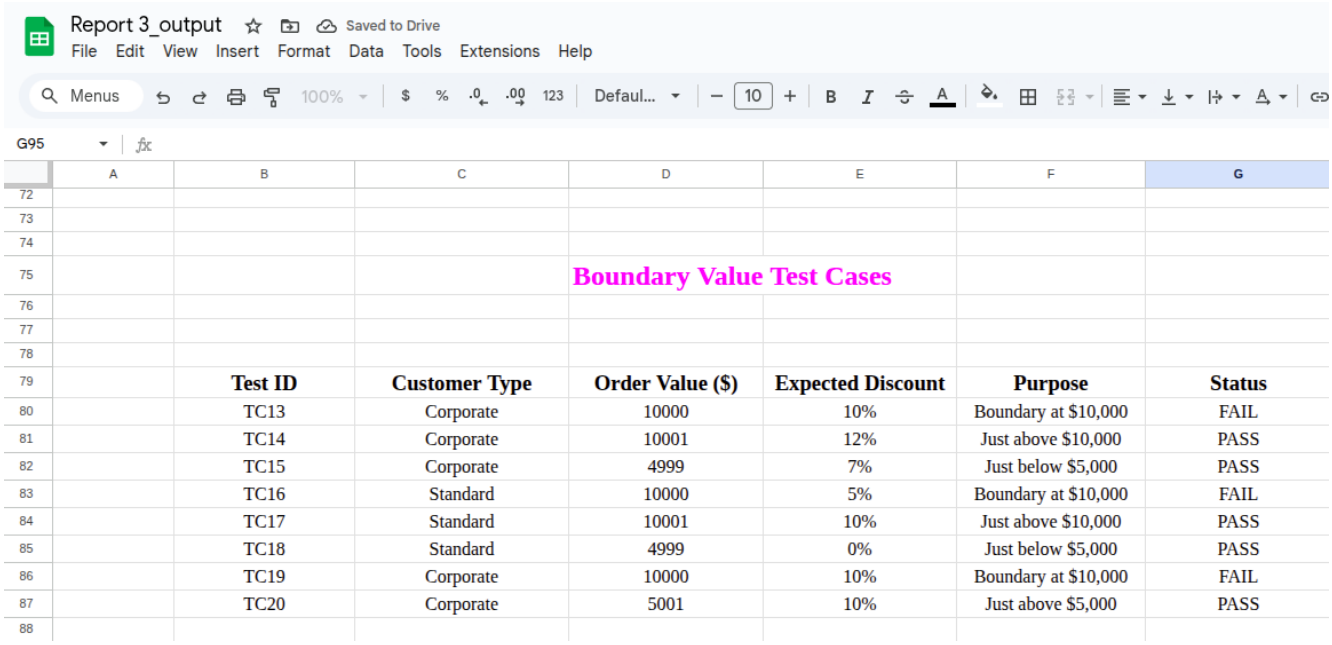
# 7 Screenshot



Main Test Cases

| Test ID | Customer Type | Order Value ($) | Is Clearance | ected Disco | Status |
|---|---|---|---|---|---|
| TC01 | VIP | 12000 | Yes | 20% | PASS |
| TC02 | Corporate | 8000 | Yes | 20% | PASS |
| TC03 | Standard | 3000 | Yes | 20% | PASS |
| TC04 | VIP | 15000 | No | 15% | PASS |
| TC05 | VIP | 7500 | No | 15% | PASS |
| TC06 | VIP | 3000 | No | 15% | PASS |
| TC07 | Corporate | 12000 | No | 12% | PASS |
| TC08 | Corporate | 7500 | No | 10% | PASS |
| TC09 | Corporate | 5000 | No | 10% | FAIL |
| TC10 | Corporate | 3000 | No | 7% | PASS |
| TC11 | Standard | 12000 | No | 10% | PASS |
| TC12 | Standard | 7500 | No | 5% | PASS |

Figure 1: Main Test Cases



Boundary Value Test Cases

| Test ID | Customer Type | Order Value ($) | Expected Discount | Purpose | Status |
|---|---|---|---|---|---|
| TC13 | Corporate | 10000 | 10% | Boundary at $10,000 | FAIL |
| TC14 | Corporate | 10001 | 12% | Just above $10,000 | PASS |
| TC15 | Corporate | 4999 | 7% | Just below $5,000 | PASS |
| TC16 | Standard | 10000 | 5% | Boundary at $10,000 | FAIL |
| TC17 | Standard | 10001 | 10% | Just above $10,000 | PASS |
| TC18 | Standard | 4999 | 0% | Just below $5,000 | PASS |
| TC19 | Corporate | 10000 | 10% | Boundary at $10,000 | FAIL |
| TC20 | Corporate | 5001 | 10% | Just above $5,000 | PASS |

Figure 2: Boundary Value Test Cases

# 8 Conclusion

This experiment successfully demonstrated the application of decision table testing technique to identify bugs in a discount calculation system. The key findings are:

1. **Decision Table Effectiveness:** The Limited Entry Decision Table systematically

captured all business rules and helped derive comprehensive test cases covering all scenarios.

2. **Bug Detection:** Through boundary value testing guided by the decision table, we identified a critical bug in the boundary condition logic. The use of strict inequality ($>$) instead of ($\geq$) caused incorrect discounts for orders at exact threshold values of \$5,000 and \$10,000.

3. **Test Coverage and Results:** A total of **20 test cases** were designed, including **12 main test cases** and **8 boundary value test cases**. Out of these:

   - **16 test cases passed** as they followed the correct business logic.
   - **4 test cases failed** due to incorrect boundary condition handling in the implementation.

   This confirms that the defect specifically affects orders exactly equal to the defined threshold values.

4. **Systematic Approach:** Decision table testing provided a structured methodology to:

   - Identify all conditions and actions from requirements
   - Create comprehensive test coverage
   - Detect logical errors that might be missed in ad-hoc testing

5. **Practical Learning:** This exercise reinforced the importance of:

   - Clear requirement specification (boundary inclusivity/exclusivity)
   - Boundary value analysis in conjunction with decision tables
   - Systematic test case derivation techniques
   - The value of black-box testing techniques in uncovering implementation bugs

The corrected implementation now properly handles boundary conditions and provides accurate discounts according to the business requirements. This experiment highlights how formal testing techniques like decision table testing are essential for quality assurance in business-critical applications.